

A faint, light gray background illustration of a neural network. It features multiple layers of circular nodes connected by a web of thin lines, representing the architecture of a deep learning model. The nodes are arranged in a somewhat symmetrical, funnel-like shape, with more nodes in the middle layers and fewer on the outer layers.

시퀀스 레이블링과 양방향 LSTM 아키텍처 구축

PyTorch를 활용한 개체명 인식(NER) 모델 구현 완벽 가이드

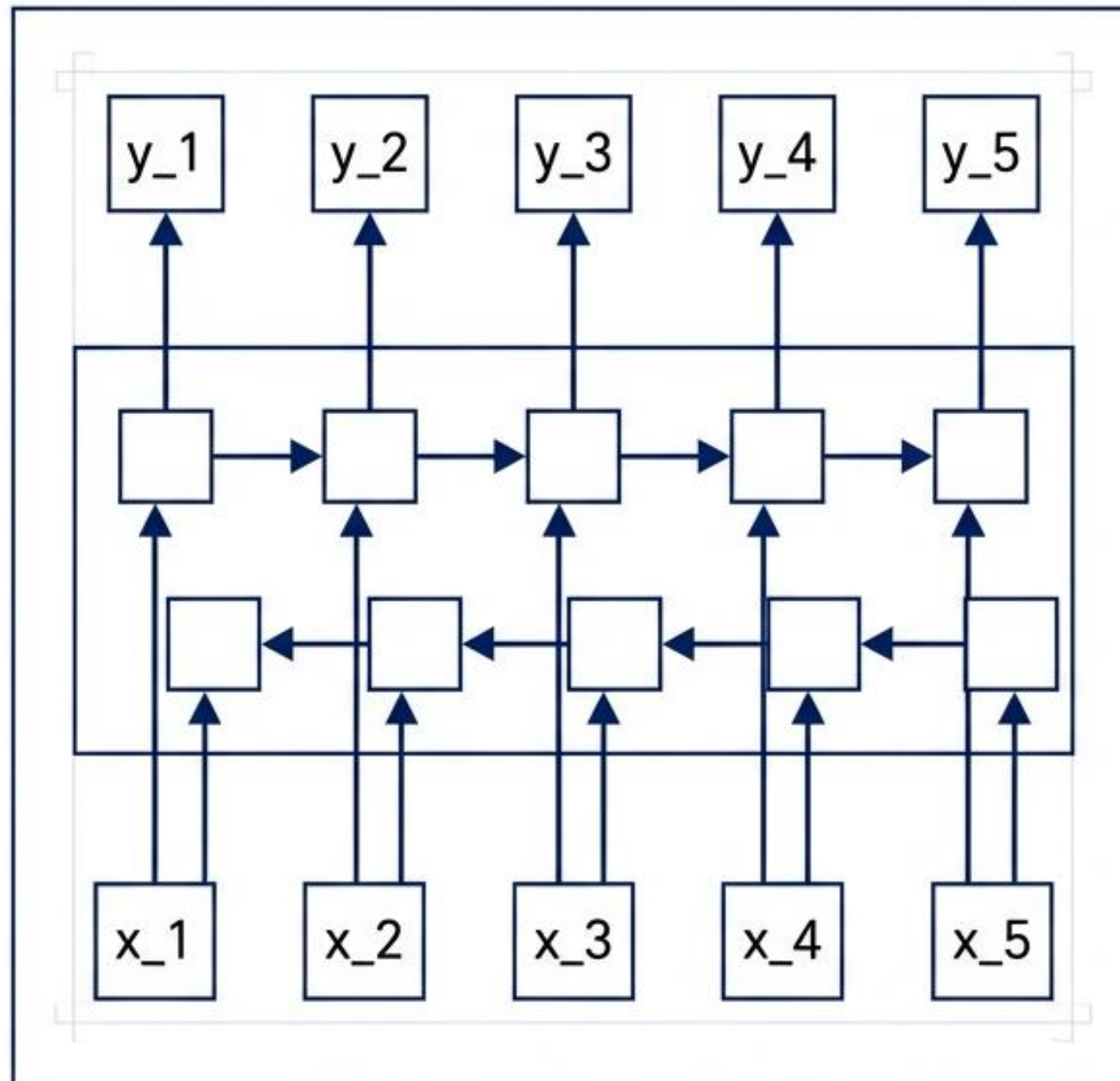
시퀀스 레이블링: 다-대-다(Many-to-Many) 신경망 설계

입력 시퀀스의 매 시점(Time Step)마다 개별적인 출력 레이블을 생성하는 자연어 처리 기법

입력 $X = [x_1, x_2, x_3, \dots, x_n]$ 에 대하여 레이블 $y = [y_1, y_2, y_3, \dots, y_n]$ 를 1:1로 부여

품사 태깅(POS Tagging) 및 개체명 인식(NER)이 대표적인 활용 사례

이전 시점뿐만 아니라 다음 시점의 문맥을 함께 파악하기 위해 양방향(Bidirectional) RNN 구조 채택 필수



지도 학습을 위한 완벽한 병렬 데이터 구조

시퀀스 레이블링은 지도 학습으로 분류되며, 하나의 샘플 내에서 단어 시퀀스(X)와 태깅 정보(y)의 길이는 태깅 정보(y)의 길이는 완벽하게 동일해야 합니다.

idx	X_train	y_train	길이
0	['EU', 'rejects', 'German', 'call', 'to', 'boycott', 'British', 'lamb']	['B-ORG', '0', 'B-MISC', '0', '0', '0', 'B-MISC', '0']	8
1	['peter', 'blackburn']	['B-PER', 'I-PER']	2
2	['brussels', '1996-08-22']	['B-LOC', '0']	2
3	['The', 'European', 'Commission']	['0', 'B-ORG', 'I-ORG']	3

인덱스 3번 샘플의 1:1 매칭 구조

개체명 인식(NER)의 정의 및 NLTK 베이스라인 검증

코퍼스 내에서 이름을 가진 개체를 식별하고 사람, 장소, 조직 등으로 분류하는 작업.
NLTK 내장 모듈(ne_chunk)을 통한 작동 원리 사전 검증.

```
from nltk import word_tokenize, pos_tag, ne_chunk

sentence = 'James is working at Disney in London'
tokenized_sentence = pos_tag(word_tokenize(sentence))
print(tokenized_sentence)

ner_sentence = ne_chunk(tokenized_sentence)
print(ner_sentence)
```

```
[('James', 'NNP'), ('is', 'VBZ'), ('working', 'VBG'), ('at', 'IN'), ('Disney', 'NNP'), ('in', 'IN'), ('London', 'NNP')]
```

```
(S
 (PERSON James/NNP)
 is/VBZ
 working/VBG
 at/IN
 (ORGANIZATION Disney/NNP)
 in/IN
 (GPE London/NNP))
```

<https://m.blog.naver.com/bycho211/221893460325>

1단계: 원시 코퍼스 로드 및 정제된 시퀀스 파싱

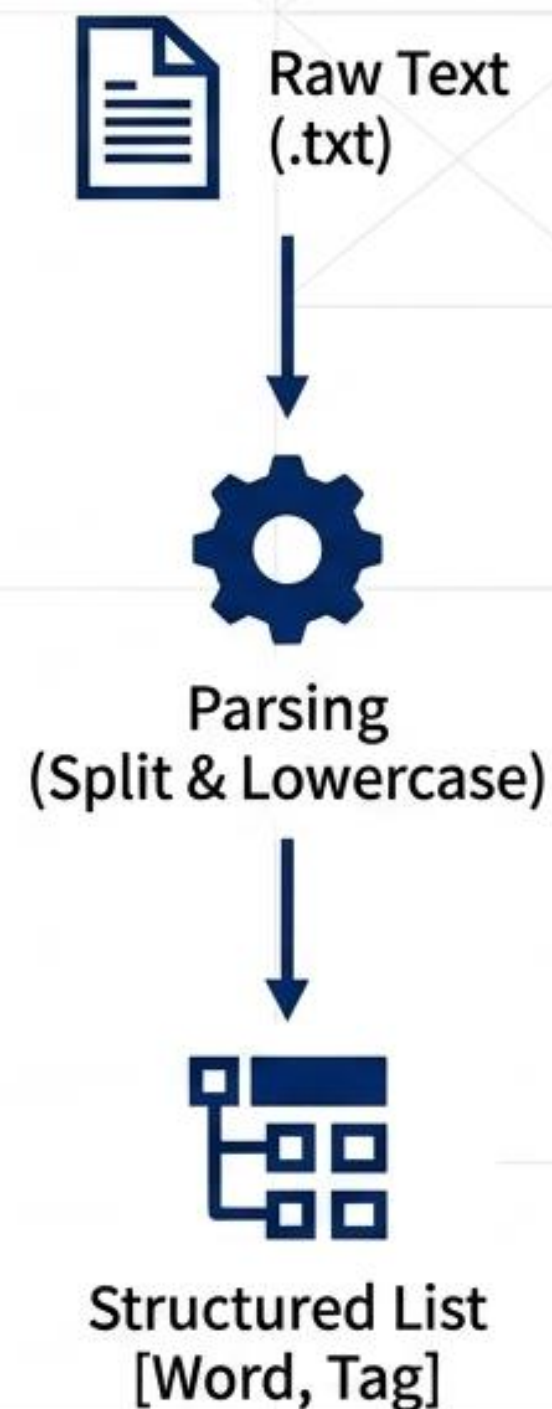
```
import urllib.request
import re

# 다운로드 및 파일 로드
urllib.request.urlretrieve('https://raw.githubusercontent.com/.../train.txt', filename='train.txt')
f = open('train.txt', 'r')

tagged_sentences = []
sentence = []
for line in f:
    if len(line)==0 or line.startswith('-DOCSTART') or line[0]=='\n':
        if len(sentence) > 0: tagged_sentences.append(sentence)
        sentence = []
        continue
    splits = line.split(' ') # 공백 기준 분리
    splits[-1] = re.sub(r'\n', '', splits[-1]) # 줄바꿈 제거
    word = splits[0].lower() # 소문자 변환
    sentence.append([word, splits[-1]])
```

전체 샘플 개수: 14041

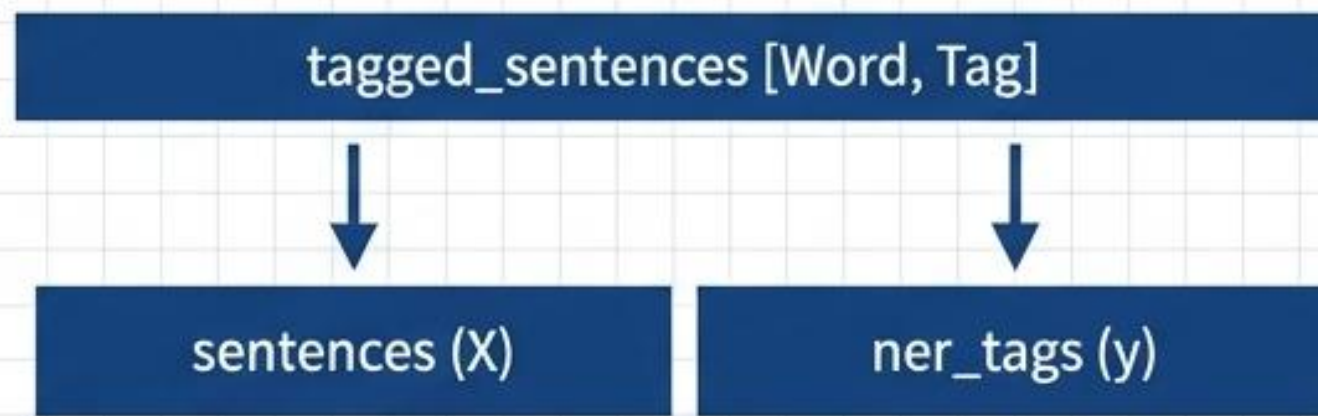
```
[['eu', 'B-ORG'], ['rejects', 'O'], ['german', 'B-MISC'], ['call', 'O'], ['to', 'O'], ['boycott', 'O'], ['british', 'B-MISC'], ['lamb', 'O'], ['.', 'O']]
```



입력/출력 데이터 분리 및 학습 파이프라인 분할

```
sentences, ner_tags = [], []  
for tagged_sentence in tagged_sentences:  
    sentence, tag_info = zip(*tagged_sentence)  
    sentences.append(list(sentence))  
    ner_tags.append(list(tag_info))  
  
from sklearn.model_selection import train_test_split  
X_train, X_test, y_train, y_test = train_test_split(  
    sentences, ner_tags, test_size=.2, random_state=777)  
  
X_train, X_valid, y_train, y_valid = train_test_split(  
    X_train, y_train, test_size=.2, random_state=777)
```

Unzipping



모델 범용성 확보를 위한 데이터 파티셔닝

2단계: 단어 집합(Vocabulary) 구축 및 특수 토큰 예약

```
from collections import Counter

word_list = [word for sent in X_train for word in sent]
word_counts = Counter(word_list)
vocab = sorted(word_counts, key=word_counts.get, reverse=True)

word_to_index = {}
word_to_index['<PAD>'] = 0 # 길이 조절용
word_to_index['<UNK>'] = 1 # OOV 문제 해결용

for index, word in enumerate(vocab):
    word_to_index[word] = index + 2
```

Index 0 : <PAD>
Index 1 : <UNK>
Index 2 : 'the'
Index 3 : ','
Index 4 : '.'
...

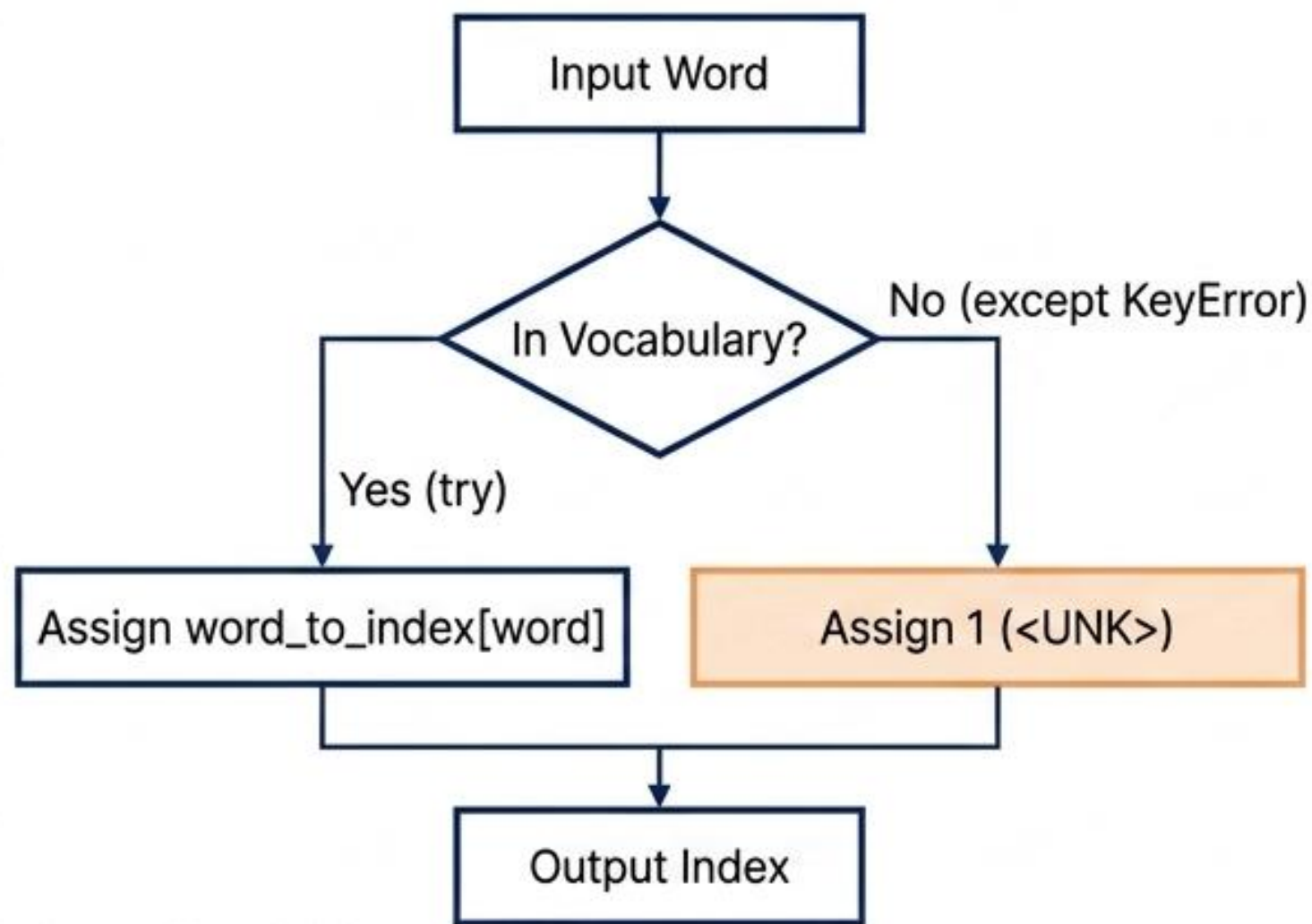
단어 집합 총
크기: 16744개

등장 빈도수 상위 10개 단어:

['the', ',', '.', 'of', 'in', 'to', 'a', ')', '(', 'and']

OOV를 고려한 X 데이터 정수 인코딩

문자열 시퀀스를 모델 연산을 위한 정수 텐서로 변환.
사전에 등록되지 않은 미등록 단어(Out-Of-Vocabulary)는
예외 처리를 통해 <UNK> 토큰으로 일괄 변환.



```
def texts_to_sequences(tokenized_X_data, word_to_index):  
    encoded_X_data = []  
    for sent in tokenized_X_data:  
        index_sequences = []  
        for word in sent:  
            try:  
                index_sequences.append(word_to_index[word])  
            except KeyError:  
                index_sequences.append(word_to_index['<UNK>'])  
        encoded_X_data.append(index_sequences)  
    return encoded_X_data
```

```
encoded_X_train = texts_to_sequences(X_train, word_to_index)  
print(encoded_X_train[:2])  
>> [[1260, 3215, 117, 17, 21, 123, 56, 539, 23],  
      [5456, 10, 8229, 9, 8230, 186, 84, 1815, 11, 8...]]
```


타겟 태그(Label) 집합 구축 및 y 시퀀스 인코딩

```
flatten_tags = [tag for sent in y_train for tag in sent]
tag_vocab = list(set(flatten_tags))

tag_to_index = {'<PAD>': 0}
for index, word in enumerate(tag_vocab):
    tag_to_index[word] = index + 1

def encoding_label(sequence, tag_to_index):
    return [[tag_to_index[tag] for tag in seq] for seq in sequence]
encoded_y_train = encoding_label(y_train, tag_to_index)
```

태그 집합: {'<PAD>': 0,
'B-PER': 1, 'I-MISC': 2,
'B-ORG': 3, 'I-PER': 4,
'B-LOC': 5, 'I-LOC': 6,
'I-ORG': 7, 'O': 8,
'B-MISC': 9}

encoded_X_train[0]

1260	3215	117	17	21	123	56	539	23
------	------	-----	----	----	-----	----	-----	----

encoded_y_train[0]

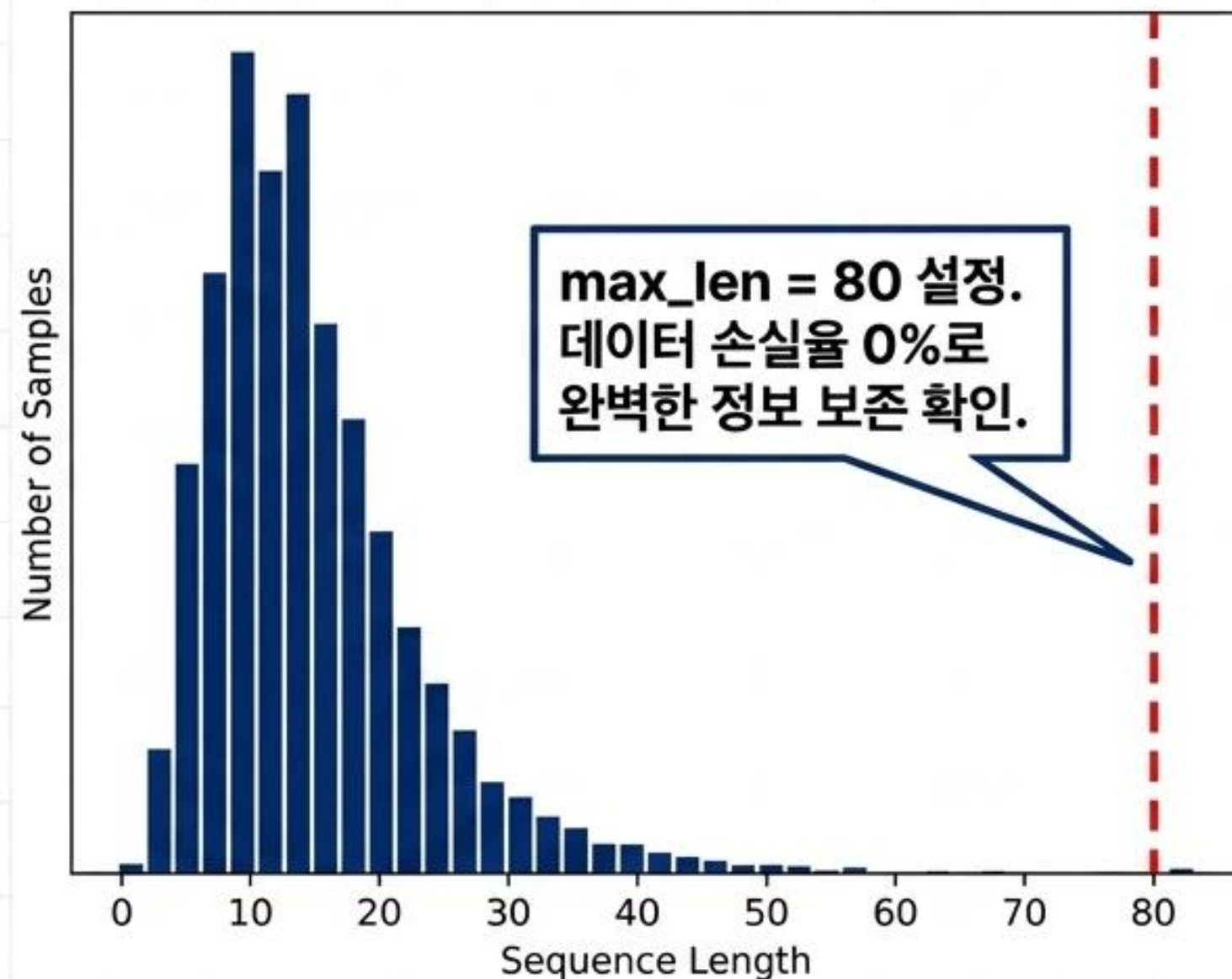
3	7	8	8	8	8	8	8	8
---	---	---	---	---	---	---	---	---

완벽한 시퀀스
병렬 구조 유지
(길이 9 일치)

3단계: 길이 분포 분석 및 임계 길이(Max Length) 도출

```
print('최대 길이 : %d' % max(len(l) for l in  
encoded_X_train))  
print('평균 길이 : %f' % (sum(map(len,  
encoded_X_train))/len(encoded_X_train)))  
  
def below_threshold_len(max_len, nested_list):  
    count = sum(1 for sentence in nested_list if  
len(sentence) <= max_len)  
    print('길이가 %s 이하인 비율: %s' % (max_len,  
(count / len(nested_list))*100))  
  
below_threshold_len(80, encoded_X_train)
```

최대 길이 : 78
평균 길이 : 14.518420
전체 샘플 중 길이가 80 이하인 비율: 100.0



고정 길이 패딩(Padding) 및 텐서 형상 동기화

```
def pad_sequences(sentences, max_len):  
    features = np.zeros((len(sentences), max_len), dtype=int)  
    for index, sentence in enumerate(sentences):  
        if len(sentence) != 0:  
            features[index, :len(sentence)] = np.array(sentence)[:max_len]  
    return features
```

```
padded_X_train = pad_sequences(encoded_X_train, max_len=80)  
padded_y_train = pad_sequences(encoded_y_train, max_len=80)
```

훈련 데이터 X 크기 : (8985, 80)
훈련 데이터 y 크기 : (8985, 80)

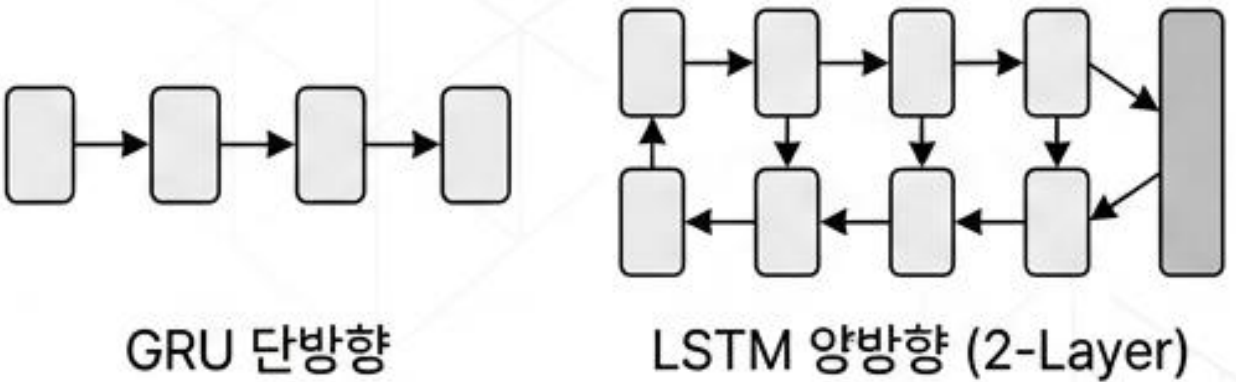
빈 공간을 0(<PAD>)으로 이식하여 모든 시퀀스를
균일한 2차원 행렬(8985x80)로 조립

[1260, 3215, 117...]	0	0	0	0	0	0	0	0	0
[1260, 3215, 117...]	0	0	0	0	0	0	0	0	0
[1260, 3215, 117...]	0	0	0	0	0	0	0	0	0
[1260, 3215, 117...]	0	0	0	0	0	0	0	0	0
[1260, 3215, 117...]	0	0	0	0	0	0	0	0	0

max_len=80

4단계: 모델 아키텍처 비교 분석

(GRU vs Bi-LSTM)



구조적 특징	단방향 GRU (Baseline)	양방향 LSTM (Selected)	아키텍처 개선 효과
Layer 클래스	<code>nn.GRU</code>	<code>nn.LSTM</code>	시퀀스 장기 의존성(Long-term dependency) 강화
심도 (Depth)	1 Layer (기본값)	<code>num_layers = 2</code> 추가	비선형적 패턴 추출 성능 고도화
문맥 방향성	정방향 연산	<code>bidirectional = True</code>	문맥의 전후(앞/뒤) 단어 정보를 동시 참조
출력층 차원	<code>hidden_dim</code>	<code>hidden_dim * 2</code>	순방향과 역방향의 은닉 상태 병합으로 정보량 2배 확장

양방향 다층 LSTM 모델(NERTagger) 구현 클래스

Device: CUDA



```
class NERTagger(nn.Module):
    def __init__(self, vocab_size, embedding_dim, hidden_dim, output_dim, num_layers=2):
        super(NERTagger, self).__init__()
        self.embedding = nn.Embedding(vocab_size, embedding_dim)

        self.lstm = nn.LSTM(embedding_dim, hidden_dim, num_layers=num_layers,
                             batch_first=True, bidirectional=True)

        self.fc = nn.Linear(hidden_dim*2, output_dim)

    def forward(self, x):
        embedded = self.embedding(x)
        lstm_out, _ = self.lstm(embedded)
        logits = self.fc(lstm_out)
        return logits
```

핵심 옵션: 양방향
문맥 동시 흡수

차원 확장: 정방향과 역방향의 은닉 상태
병합으로 출력 차원 2배 증가

파이토치 TensorDataset 변환 및 학습 하이퍼파라미터 구성

텐서 변환 및 데이터로더 생성

```
X_train_tensor = torch.tensor(padded_X_train, dtype=torch.long)
y_train_tensor = torch.tensor(padded_y_train, dtype=torch.long)
```

```
train_dataset = torch.utils.data.TensorDataset(X_train_tensor, y_train_tensor)
train_dataloader = torch.utils.data.DataLoader(
    train_dataset, shuffle=True, batch_size=32)
```

모델 객체 및 하이퍼파라미터 선언

```
model = NERTagger(vocab_size=16744, embedding_dim=100, hidden_dim=256,
                  output_dim=10, num_layers=2).to(device)
```

손실 함수 및 옵티마이저

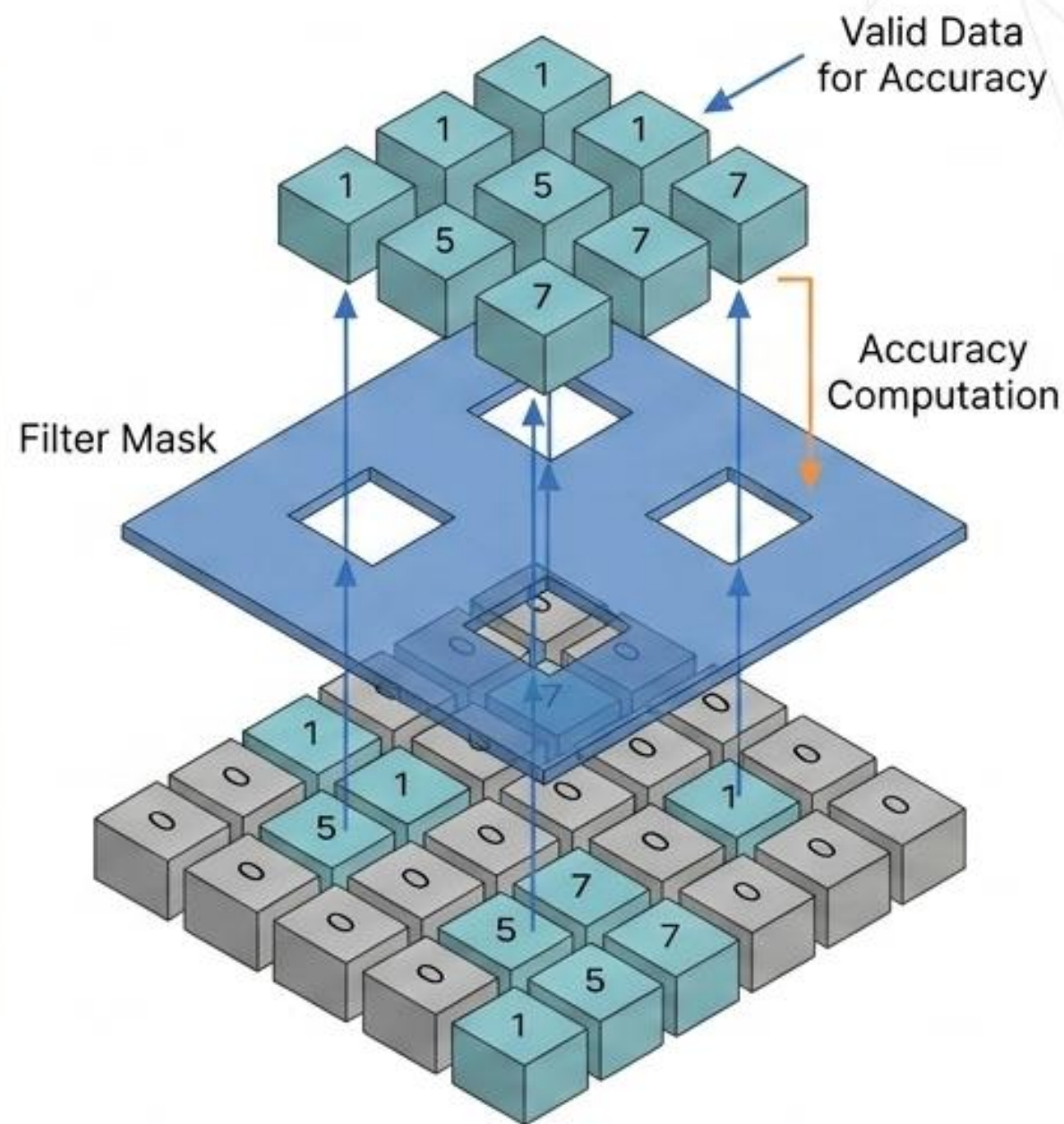
```
criterion = nn.CrossEntropyLoss(ignore_index=0)
optimizer = optim.Adam(model.parameters(), lr=0.01)
```



중요: 패딩 토큰(0)의 손실
계산을 강제 배제하여
가짜 노이즈가 역전파되는
것을 원천 차단함

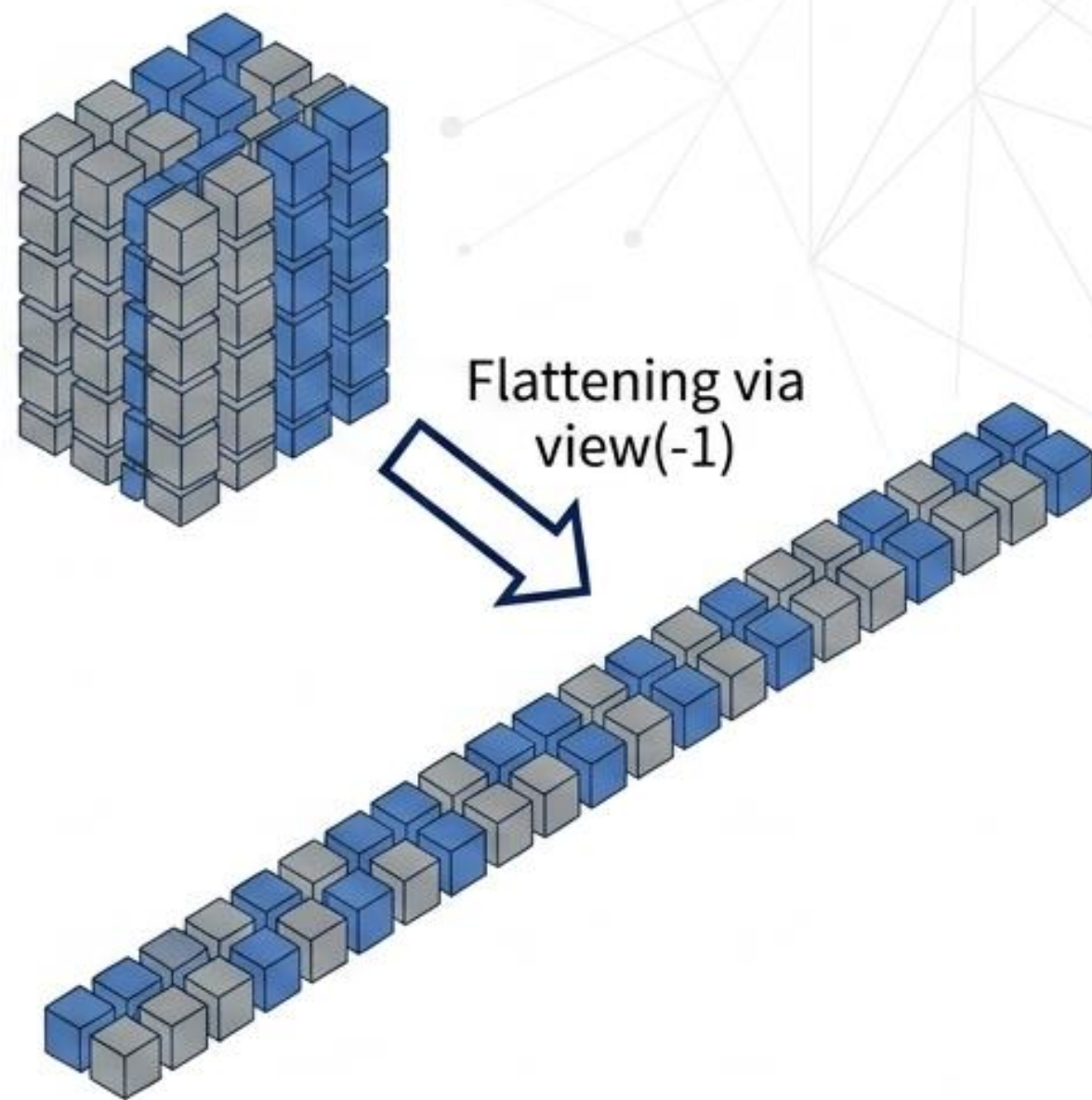
5단계: 패딩 노이즈를 필터링하는 검증(Accuracy) 로직

```
def calculate_accuracy(logits, labels, ignore_index=0):  
    # 가장 높은 확률을 가진 클래스 도출  
    predicted = torch.argmax(logits, dim=1)  
  
    # 레이블이 0(패딩)이 아닌 위치만 True로 마스크  
    mask = (labels != ignore_index)  
  
    # 마스크가 씌워진 영역 내에서 일치하는 경우만 합산  
    correct = (predicted == labels).masked_select(mask).sum().item()  
    total = mask.sum().item()  
  
    return correct / total
```



가중치 고정 기반의 성능 평가(Evaluate) 루프 설계

```
def evaluate(model, valid_dataloader, criterion, device):  
    val_loss, val_correct, val_total = 0, 0, 0  
    model.eval() # 평가 모드  
    with torch.no_grad(): # 기울기 연산 비활성화  
        for batch_X, batch_y in valid_dataloader:  
            batch_X, batch_y = batch_X.to(device), batch_y.to(device)  
            logits = model(batch_X)  
  
            # 차원 평탄화 후 연산  
            loss = criterion(logits.view(-1, 10), batch_y.view(-1))  
            val_loss += loss.item()  
  
            val_correct += calculate_accuracy(  
                logits.view(-1, 10), batch_y.view(-1)) * batch_y.size(0)  
            val_total += batch_y.size(0)  
  
    return val_loss / len(valid_dataloader), val_correct / val_total
```



능동적 모델 가중치 최적화 및 체크포인트(Checkpoint) 저장

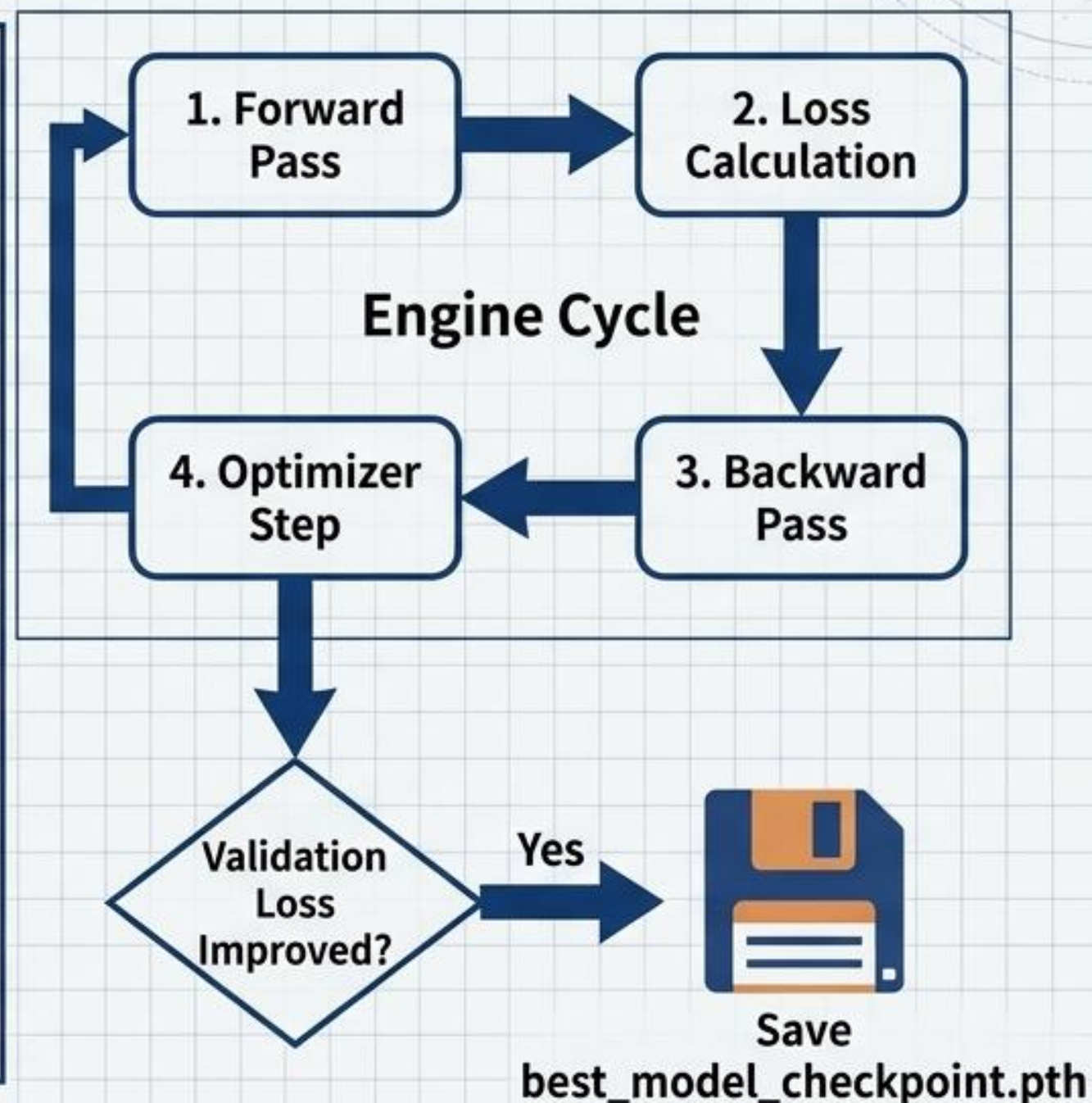
```
best_val_loss = float('inf')

for epoch in range(num_epochs):
    model.train() # 학습 모드
    for batch_X, batch_y in train_dataloader:
        batch_X, batch_y = batch_X.to(device), batch_y.to(device)
        logits = model(batch_X)
        loss = criterion(logits.view(-1, 10), batch_y.view(-1))

        optimizer.zero_grad()
        loss.backward() # 역전파
        optimizer.step() # 파라미터 업데이트

    # 검증 및 성능 평가
    val_loss, val_accuracy = evaluate(model, valid_dataloader,
                                      criterion, device)

    # 모델 가중치 최적화 및 저장
    if val_loss < best_val_loss:
        best_val_loss = val_loss
        torch.save(model.state_dict(), 'best_model_checkpoint.pth')
```



최적 가중치 복원 및 테스트 셋(Unseen Data) 성능 최종 평가

```
# 최적 모델 가중치 로드
```

```
model.load_state_dict(torch.load('best_model_checkpoint.pth'))
```

```
model.to(device)
```

```
val_loss, val_accuracy = evaluate(model, valid_dataloader, criterion, device)
```

```
test_loss, test_accuracy = evaluate(model, test_dataloader, criterion, device)
```

Validation Accuracy

0.9560

Unseen Test Accuracy

95.66%

학습 과정에서 관측되지 않은 데이터에 대한 탁월한 범용 추론 능력 검증 완료

6단계: 실전 인퍼런스(Inference)를 위한 전처리 역산 파이프라인

predict_labels 함수 캡슐화(Encapsulation) 영역



인퍼런스 파이프라인(predict_labels) 통합 함수 구현

```
index_to_tag = {value: key for key, value in tag_to_index.items()}

def predict_labels(text, model, word_to_ix, index_to_tag, max_len=150):
    tokens = text.split()
    token_indices = [word_to_ix.get(token, 1) for token in tokens]

    token_indices_padded = np.zeros(max_len, dtype=int)
    token_indices_padded[:len(token_indices)] = token_indices[:max_len]
    input_tensor = torch.tensor(token_indices_padded, dtype=torch.long).unsqueeze(0).to(device)

    model.eval()
    with torch.no_grad():
        logits = model(input_tensor)
        predicted_indices = torch.argmax(logits, dim=-1).squeeze(0).tolist()

    predicted_indices_no_pad = predicted_indices[:len(tokens)]
    return [index_to_tag[idx] for idx in predicted_indices_no_pad]
```

predict_labels 함수 캡슐화
(Encapsulation) 영역

1. Raw Text
Input
(임의 문자열 입력)

2. Tokenization
& Encoding
(공백 분리 및 사전 매핑)

3. Padding
(max_len 0 강제 채움)

4. Tensorization
& Forward Pass
(배치 확장 및 모델 연산)

5. Argmax &
Truncation
(최고 확률 추출 및
패딩 제거)

6. Decoding
(정수를 인간이 읽을 수
있는 태그로 변환)

아키텍처 완성: 인퍼런스 실전 예측 결과 분석

Raw Input (Unseen Test Data)

feyenoord rotterdam suffered an early shock when they went 1-0 down after four minutes against de graafschap doetinchem .

예측 : ['B-ORG', 'I-ORG', '0', '0', '0', '0', '0', '0', '0', '0', '0', '0', '0', '0', 'B-ORG', 'I-ORG', 'I-ORG', '0']

실제값 : ['B-ORG', 'I-ORG', '0', '0', '0', '0', '0', '0', '0', '0', '0', '0', '0', '0', '0', '0', 'B-ORG', 'I-ORG', '0']

설계도면에서 시작된 언어 인프라 구축의 성공적 완성. 완벽한 조직명(ORG) 인식 검증.